```
import itertools

# Define custom functions for Propositional Logic operators
def negation(p):
    return not p

def conjunction(p, q):
    return p and q

def disjunction(p, q):
    return p or q

def exclusive_or(p, q):
    return p ^ q

def implication(p, q):
    return not p or q

def biconditional(p, q):
    return p == q

def evaluate_and_print_table():
    # 1. Setup the header for the truth table
    print(f"{'P':<6} | {'Q':<6} | {'¬P':<6} | {'P ∧ Q':<6} | {'P ∨ Q':<6} | {'P ⊕ Q':<6} | {'P → Q':<6} | {'P ↔ Q':<6}")
    print("-" * 64)

    # 2. Generate all combinations of truth values for P and Q
    boolean_values = [True, False]
    combinations = list(itertools.product(boolean_values, repeat=2))

    # 3. Evaluate operators for each combination
    for p, q in combinations:
        neg = negation(p)
        conj = conjunction(p, q)
        disj = disjunction(p, q)
        xor = exclusive_or(p, q)
        impl = implication(p, q)
        bico = biconditional(p, q)

        # Format output using 1 for True and 0 for False for clean presentation
        print(f"{int(p):<6} | {int(q):<6} | {int(neg):<6} | {int(conj):<6} | {int(disj):<6} | {int(xor):<6} | {int(impl):<6} | {

    print("\n" + "="*50 + "\n")

    # 4. Evaluating a complex compound statement: (P → Q) ∧ (Q → P)
    print("Evaluating Compound Statement: (P ∨ Q) ∧ (Q → P)")
    print(f"{'P':<6} | {'Q':<6} | {'(P → Q) ∧ (Q → P)':<20}")
    print("-" * 38)

    results = []
    for p, q in combinations:
        # Compound formula calculation
        expr = implication(p, q) and implication(q, p)
        results.append(expr)
        print(f"{int(p):<6} | {int(q):<6} | {int(expr):<20}")

    # 5. Determine Statement Classification (Tautology, Contradiction, or Satisfiable)
    if all(results):
        classification = "Tautology (Always True)"
    elif not any(results):
        classification = "Contradiction (Always False)"
    else:
        classification = "Satisfiable (Contingency)"

    print(f"\nClassification of the compound expression: {classification}")

if __name__ == "__main__":
    evaluate_and_print_table()
```

```
P      | Q      | ¬P     | P ∧ Q  | P ∨ Q  | P ⊕ Q  | P → Q  | P ↔ Q
----------------------------------------------------------------
1      | 1      | 0      | 1      | 1      | 0      | 1      | 1
1      | 0      | 0      | 0      | 1      | 1      | 0      | 0
0      | 1      | 1      | 0      | 1      | 1      | 1      | 0
```

```
0      | 0      | 1      | 0      | 0      | 0      | 1      | 1

================================================

Evaluating Compound Statement: (P → Q) ∧ (Q → P)
P      | Q      | (P → Q) ∧ (Q → P)
---------------------------------------
1      | 1      | 1
1      | 0      | 0
0      | 1      | 0
0      | 0      | 1

Classification of the compound expression: Satisfiable (Contingency)
```